CORE JAVA SERIES — DAY 1

Why Was Java Born?

Java's origin story, explained from first principles — from the problems with C/C++ to Bytecode, the JVM, JRE, and JDK.

Before Java: C and C++ (1980s–90s)

Back then, C and C++ were the dominant languages. The reasons for their popularity:

- Very **fast**
- **Simpler** compared to older languages
- **Low-level** — very close to hardware, few abstraction layers

> **Abstraction:** When a language handles a lot internally and you just give a "plain English" instruction — like Python. C/C++ had little abstraction; you had to implement most things yourself.

Problem 1: Portability

Everything was going fine, but there was one big problem: **Portability** — running the same source code on different platforms.

Role of a Compiler

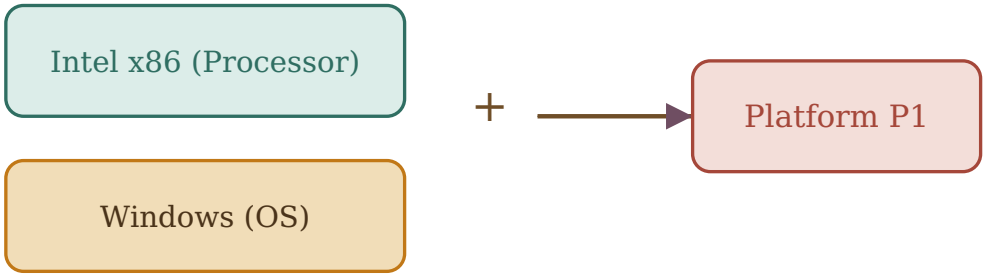
A computer only understands binary (0s and 1s / machine code). A **compiler** is software that converts your language's source code into machine code.



Source code → Compiler → Machine code. This machine code is specific to one particular platform.

What Is a Platform?

Platform = **Processor** + **Operating System** combined.



Similarly, ARM + macOS together form a Platform P2. Every platform is different.

Now if the same `hello.cpp` has to run on both P1 and P2, **different machine code** gets generated — meaning it must be compiled separately for each platform. Two reasons:

Reason A — Operating System

Printing to console, reading/writing files, allocating memory — for all of this, a program talks to the OS. Every OS provides its own **system libraries** (in compiled form). Windows's methods differ from macOS's — so the final binary differs too.

Reason B — Processor (ISA)

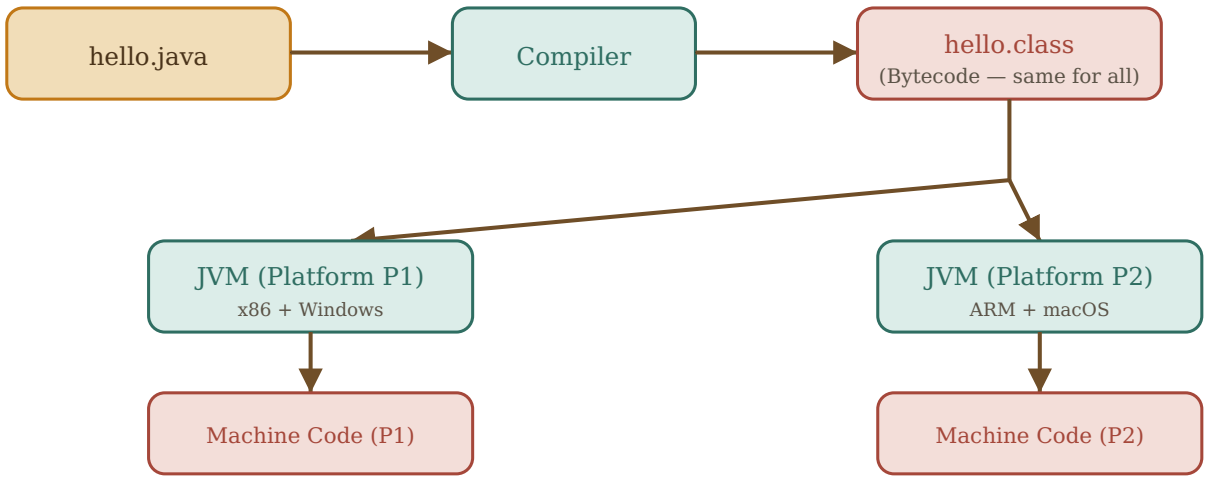
Inside a processor are billions/trillions of **transistors** (current flowing = 1, not flowing = 0). Every processor (Intel vs ARM) has its own **ISA (Instruction Set Architecture)** — this is the processor's "grammar" for how small commands like add/load/store execute. Different processor = different ISA = different binary.

> **Result:** C/C++ (and every language before it) was **platform-dependent**. Every new platform required recompiling from scratch.

Java's Solution: Bytecode + JVM

Instead of generating machine code directly, Java created an intermediate code called **Bytecode** (.class file). This bytecode stays the same across every platform.

> **Analogy:** You only know Hindi/English. Before visiting China, you bring along a translator friend who knows every language. You speak in Hindi, and they convert it to Mandarin for you. That translator is the **JVM**.



Once the .class file (bytecode) exists, each platform's own JVM reads it and converts it into that platform's machine code.

> **Important correction:** The JVM itself is **not platform-independent** — a separate JVM has to be written for each platform (just like a different translator for each country). Only **Bytecode** is platform-independent. The JVM must know that platform's OS and processor ISA.

> **WORA = Write Once, Run Anywhere.** Compile once to produce bytecode, then run it wherever a JVM is installed — no need to recompile.

Why Was Portability So Important?

- In the 1980s–90s, new embedded systems/devices (set-top boxes, TVs) were emerging — the same application had to run on all of them
- The internet was starting up — the same code had to be deployed on multiple servers (different platforms)
- Processors had already become fast, so "speed" mattered less — a general-purpose language became more important

Java's Three Pillars

Portable	Simple	Secure
Via Bytecode + JVM	Removed pointers, multiple inheritance, manual memory deallocation	Via the JVM's Sandbox model

Simplicity

Certain things in C/C++ added complexity: **pointers**, **multiple inheritance**, and **manual memory allocation/deallocation** (especially deallocation). Java removed all three so the language would be easier for everyone to learn.

Security

Java was adopted extremely fast — on servers (**Java Servlets**) and inside browsers (**Java Applets**, even before JavaScript existed). Applets would download and run in the browser the moment a link was clicked — but this created a risk of misuse (a hacker could gain permissions on the system, steal data, and so on).



Sandbox Model: the JVM runs bytecode inside a restricted environment — no harmful or unwanted permission can be granted.

Interesting: Portability and Security are both solved by the **JVM** alone. Simplicity comes from Java's own syntax design.

Fun Facts

- **JavaScript** was named after Java (for marketing purposes) — the two have no technical relation. Java existed before JavaScript.
- **Microsoft** tried this same "bytecode + virtual machine" experiment with C/C++, and it failed (because C/C++'s whole purpose was to stay hardware-close and fast). So instead they created a new language, **C#**, which is platform-independent — an alternative to Java.
- Java Applets were discontinued around JDK 10/11 — JavaScript is used on the front end now.

Active Recall — Part 1

Q1. What is a platform?

A. A combination of Processor + Operating System (e.g. Intel x86 + Windows).

Q2. Why can't C/C++ code compiled for one platform run on another?

A. Because the OS's system libraries differ and the processor's ISA (Instruction Set Architecture) differs — so different machine code (binary) is generated.

Q3. What is bytecode, and what file extension is it saved with?

A. An intermediate code that isn't machine code — identical across all platforms. File extension: `.class`.

Q4. Is the JVM itself platform-independent?

A. No. The JVM is platform-dependent (a separate JVM is built for each platform); only bytecode is platform-independent.

Q5. What does WORA stand for and mean?

A. Write Once, Run Anywhere — compile once, then run it on any platform that has a JVM.

Q6. What are Java's three pillars?

A. Portability, Simplicity, and Security.

Q7. What does Java's Sandbox Model do?

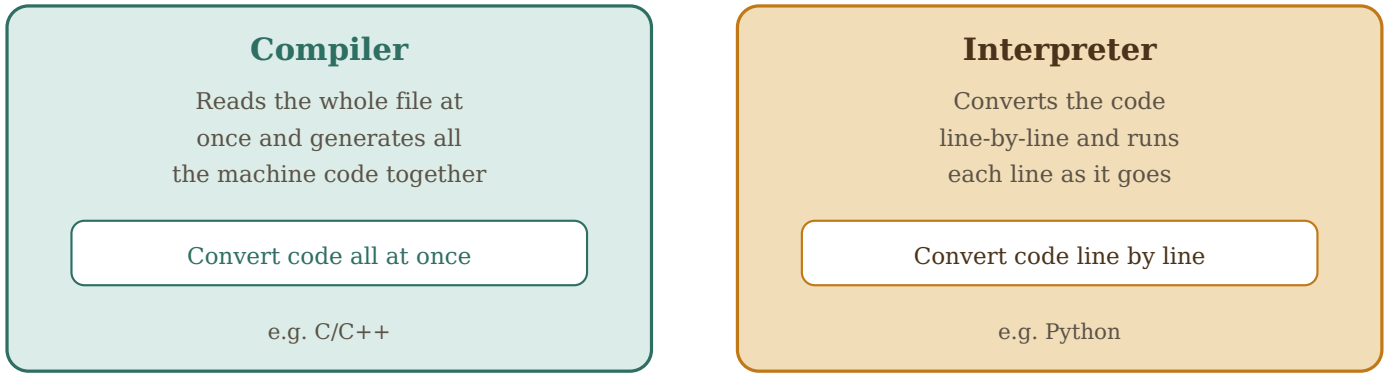
A. The JVM runs bytecode inside a restricted/controlled environment so no harmful or unwanted system access can be granted.

JVM, JRE, and JDK — Plus Compiler vs Interpreter

Now let's look at how the JVM actually works internally, whether Java is compiled or interpreted, and how the JVM → JRE → JDK hierarchy is built.

Compiler vs Interpreter

Both do the same job: converting source code (or bytecode) into machine code. The difference is in how they do it:

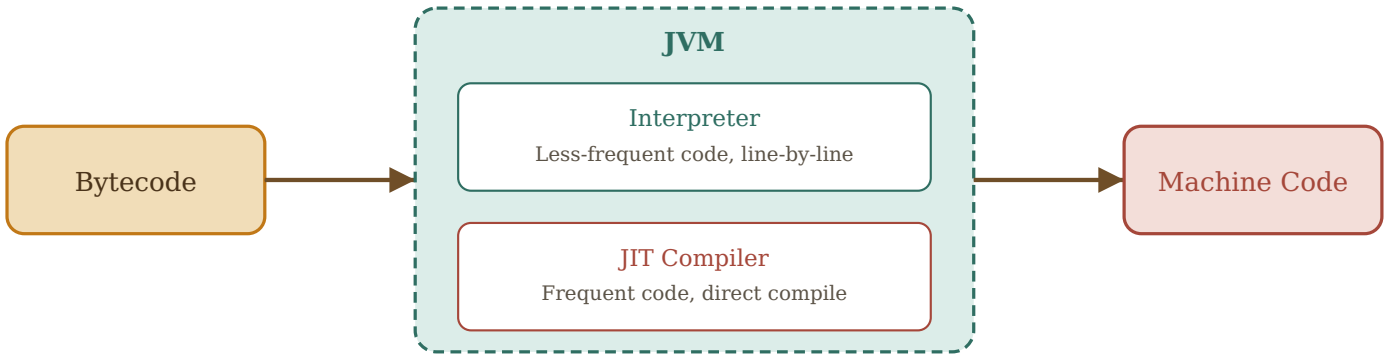


Compiler = converts the whole code at once; Interpreter = converts and runs line by line.

Java is Both: Java is both compiled and interpreted. First, a **compiler** converts the source code into bytecode (.class). Then, inside the JVM, an **interpreter** converts that bytecode into machine code line by line and runs it.

The JIT Compiler — A Hybrid With the Interpreter

In the 1990s, hardware was slow (slow RAM, slow disk) — so Java initially used only an **interpreter**, so a program could start running as quickly as possible. As hardware got faster, Java added a **JIT (Just-In-Time) Compiler** — it doesn't replace the interpreter, it works alongside it.



Frequently-used code is compiled directly by the JIT Compiler; the rest is handled line-by-line by the Interpreter. Together they turn bytecode into machine code.

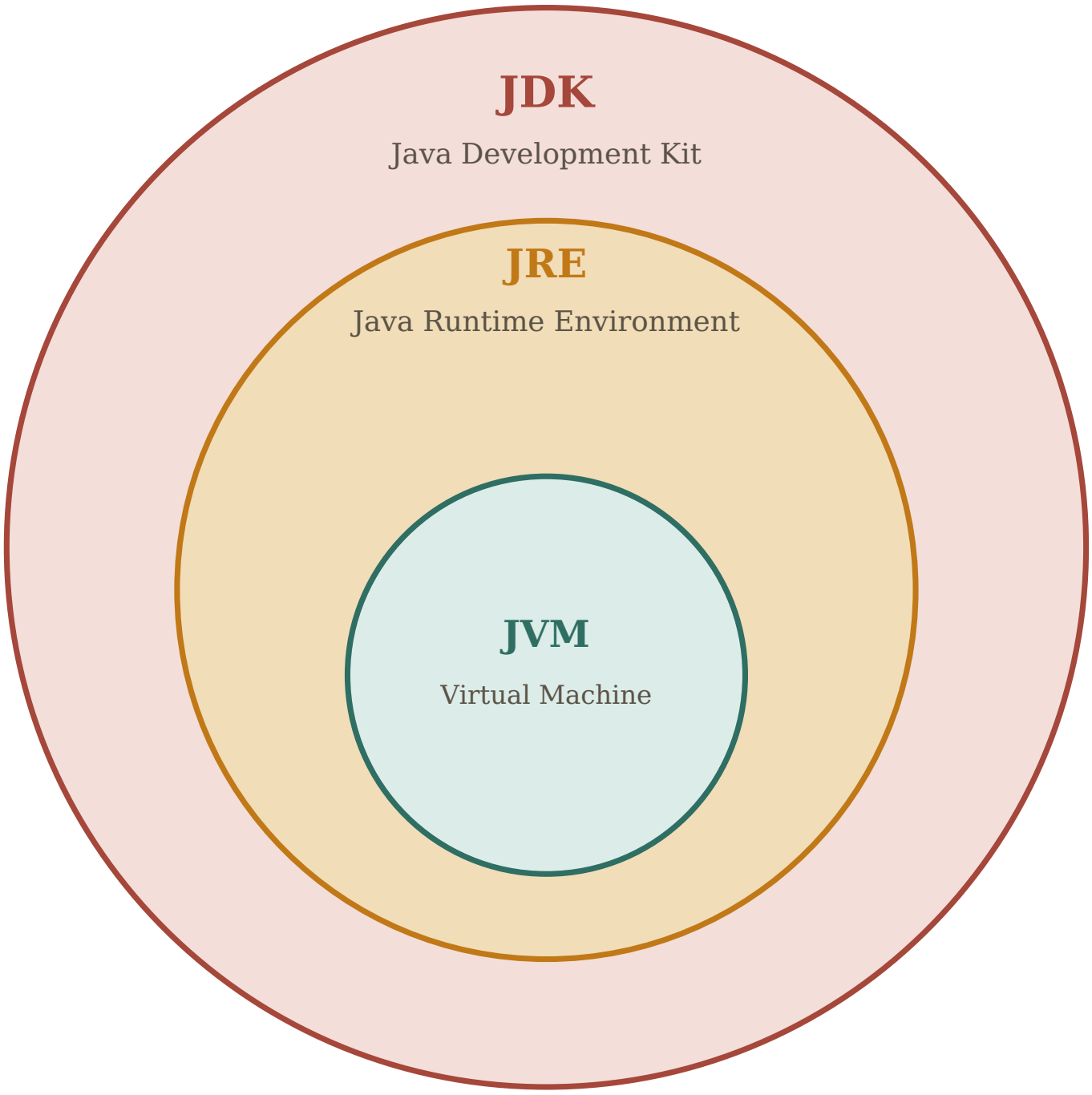
Trade-off: This extra step (bytecode → machine code) makes Java a bit slower than C/C++ at runtime — in exchange, it gets portability, simplicity, and security. On modern hardware this difference is now negligible.

The Full Job of the JVM

- **Bytecode → Machine Code:** using the Interpreter + JIT Compiler
- **Security:** the Sandbox Model — runs bytecode inside a restricted environment
- **Garbage Collection:** handles memory management automatically (covered in detail later)

JVM → JRE → JDK: The Hierarchy

These three terms sit inside one another — like concentric circles.



The JDK is the largest — it contains the JRE, which contains the JVM.

JRE = JVM + Class Libraries

Class Libraries are the built-in methods/functions a language provides internally — like printing to console, or reading/writing files. A JVM alone isn't enough to properly run a Java program — class libraries are also needed. So **JRE = JVM + Class Libraries**.

JDK = JRE + Compiler + Debugger + Dev Tools

With just the JRE, you can **run** a program, but you can't **write and compile** one — that needs a compiler. The

JDK includes the JRE plus a **Compiler** (source → bytecode), a **Debugger** (for stepping through code line by line), and tools like Javadoc for documentation.

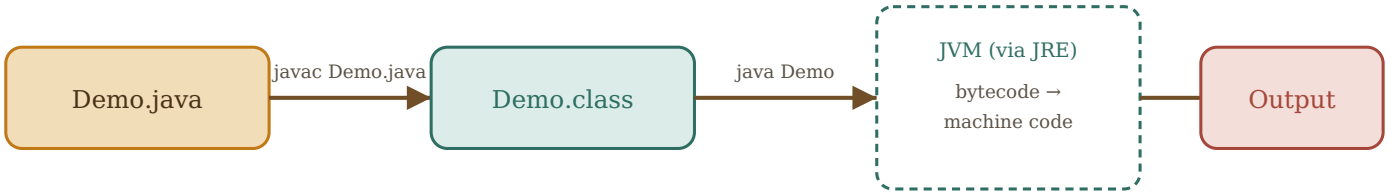
> **Practical rule:** To both write and run Java, your system needs the **JDK** installed — the JVM or JRE alone isn't enough.

Java's Editions: JSE, JEE, JME

JSE	JEE	JME
Java Standard Edition — "Core Java". OOP, classes, methods — all the basics live here.	Java Enterprise Edition (now Jakarta EE) — extra libraries for building web apps and websites.	Java Micro Edition — a lightweight edition for old-generation mobile phones. Now largely obsolete, replaced by Android.

Compiling and Running a Java Program

Once the JDK is installed, any `.java` file can be compiled and run using two commands.



`javac` turns the source into bytecode; `java` runs that bytecode through the JVM.

Active Recall — Part 2

Q1. What's the basic difference between a Compiler and an Interpreter?

A. A Compiler converts the whole code into machine code at once; an Interpreter converts and runs the code line by line.

Q2. Is Java compiled or interpreted?

A. Both. A compiler converts source code into bytecode; then, inside the JVM, an interpreter (and JIT compiler) converts that bytecode into machine code.

Q3. What does the JIT Compiler do, and why doesn't it replace the interpreter?

A. The JIT Compiler directly compiles frequently-used code for speed; the rest (less-frequent code) is still handled line-by-line by the interpreter — the two work together.

Q4. What is the JRE?

A. JRE = JVM + Class Libraries. The minimum requirement for properly running a Java program.

Q5. What is the JDK?

A. JDK = JRE + Compiler + Debugger + other dev tools. Needed for both writing and running Java code.

Q6. What's the difference between JSE, JEE, and JME?

A. JSE = Core Java (the basics). JEE (Jakarta EE) = extra libraries for web apps/enterprise use. JME = a lightweight edition for old mobile phones, now obsolete.

Q7. What are the commands to compile and run a .java file?

A. Compile: `javac Demo.java` (produces the .class bytecode file). Run: `java Demo` (the JVM converts bytecode to machine code and runs it).